

Scientific Machine Learning: Necessity & Foundations

G rard Grigore Bousquet^{1*}

¹Marine Technology, NTNU, Trondheim, Norway

ABSTRACT

This first article belongs to a series that aim to illustrate the application of Scientific Machine Learning (SciML). Although classical Machine Learning (ML) and Deep Learning (DL) have achieved remarkable success across many domains, they exhibit important limitations when addressing scientific computing problems governed by differential equations. It is well known that a Deep Neural Network (DNN) can be trained to approximate a function $f : X \rightarrow Y$ such that $\forall x \in X, f_\theta(x) \approx y$, with $\theta \in \mathbb{R}^p$ the parameters of the neural networks (NN). Such models can be employed in a wide variety of applications, ranging from classification and regression to the numerical approximation of differential equations. However, when dealing with families of parameterized Partial Differential Equations (PDEs), this approach becomes inefficient since the network generally has to be retrained for every new parameter configuration [1][2]. Rather than directly addressing these limitations, this first article introduces the fundamental building blocks of Deep Learning through the implementation of a simple Multi-Layer Perceptron (MLP). The Two-Moons classification problem is used as a pedagogical example to present the complete workflow, from network construction and training to performance evaluation. This foundation will serve as the starting point for the subsequent articles devoted to Neural Operators (NO) and Physics-Informed Neural Operators (PINO).

Keywords: Machine Learning, Scientific Machine Learning, Partial Differential Equation learning, Operator Learning, Two-Moons Problem, Neural Networks

Remark: Some mathematical formulations adopted throughout this article may be more general than required for the Two-Moons classification problem. This choice is deliberate and aims to establish a rigorous mathematical framework that will naturally extend to the subsequent articles devoted to Neural Operators and Physics-Informed Neural Operators. On another note, the implemented scripts can be found through the following links [Two-Moons problem from scratch](#) and [Two-Moons problem using classical Torch workflow](#).

1. INTRODUCTION

The Two-Moons dataset is a classical benchmark for binary classification. Geometrically, it consists of separating two groups of points embedded in the Euclidean plane $\mathbb{R}^2$. A typical Two-Moons dataset is shown in Figure 1.

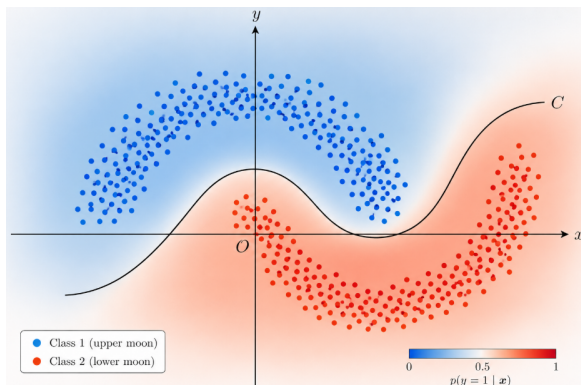


Figure 1. Two-Moons problem configuration.

The goal is to learn a function $f_\theta : (\mathbb{R}^2, \tau_u) \rightarrow \{0, 1\}$

that, given an input point $x \in \mathbb{R}^2$, predicts whether the point belongs to the lower part of the moon (red points) $f_\theta(x) = 1$ or to the upper crescent (blue points) $f_\theta(x) = 0$. In the previous picture, One possible separating curve is represented by C . τ_u denotes the Topology induced by the Euclidean distance $d_u(x, y) = \|x - y\|_2$ with $x, y \in \mathbb{R}^2$. The set τ_u can be expressed using the classical ball definition:

$$\tau_u = \{U \subset \mathbb{R}^2 | \forall x \in U, \exists r > 0; B_{d_u}(x, r) \subset U\} \quad (1)$$

With the definition of an open ball of center x and radius r :

$$B_{d_u}(x, r) = \{a | d_u(a, x) < r\} \quad (2)$$

The Universal Approximation Theorem states that feedforward neural networks with a single hidden layer are capable of approximating any continuous function on a compact domain with arbitrary accuracy, provided sufficiently many neurons are available. [3]. This theoretical result motivates the use of neural networks for nonlinear classification

*Corresponding Author: G rard Grigore Bousquet <gerard.grigore@gmail.com>

problems. The following section therefore presents the proposed neural network architecture together with the selected hyperparameters.

Although the Two-Moons dataset is unrelated to differential equations, it provides an ideal pedagogical example for introducing the fundamental concepts of supervised learning, neural network architecture, loss minimization and optimization, which will later be extended to Neural Operators and Physics-Informed Neural Operators.

2. ARCHITECTURE OF THE NN

The objective of this section is to introduce the mathematical formulation of a classical Multi-Layer Perceptron (MLP) together with its geometrical interpretation. Although the implementation considered in this work remains relatively simple, it already illustrates the fundamental mechanisms behind modern Deep Learning architectures. The network architecture used throughout this article is represented in Figure 2. For more details regarding the Perceptron algorithm and the classical NN architecture, see [3][4].

2.1. Mathematical formulation of a Multi-Layer Perceptron

As explained previously, the objective of the neural network is to learn a nonlinear function:

$$\mathbf{f}_\theta : \mathbb{R}^{d_0} \rightarrow (0, 1) \subset \mathbb{R}^{d_L} \quad (3)$$

mapping every point of the Euclidean plane to a probability describing its belonging to one of the two classes.

More generally, a fully connected neural network of depth L can be described recursively by:

$$\begin{aligned} \mathbf{x}^{(0)} &= \mathbf{x}, \\ \mathbf{x}^{(l)} &= \sigma^{(l)} \left(W^{(l)} \mathbf{x}^{(l-1)} + \mathbf{b}^{(l)} \right), \\ \mathbf{x}^{(L)} &= \sigma^{(L)} \left(W^{(L)} \mathbf{x}^{(L-1)} + \mathbf{b}^{(L)} \right). \end{aligned} \quad (4)$$

Where:

- $L \in \mathbb{N}$ denotes the depth of the network. In this work, $L = 3$.
- $\mathbf{x}^{(l)} \in \mathbb{R}^{d_l}$ denotes the output of layer l .
- $W^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$ denotes the weight matrix of layer l .
- $\mathbf{b}^{(l)} \in \mathbb{R}^{d_l}$ denotes the bias vector of layer l .
- $\sigma^{(l)}$ denotes the activation function of layer l .

Eventually, the trainable parameters of the neural network are therefore:

$$\theta = (W^{(1)} \dots W^{(L)} \mathbf{b}^{(1)} \dots \mathbf{b}^{(L)}) \quad (5)$$

The training phase consists in determining the optimal value of these parameters.

2.2. Architecture of the proposed MLP

By applying this notation to our three layers perceptron, we get the following equations and numerical values:

$$\begin{aligned} \mathbf{x}^{(0)} &= \mathbf{x}, \\ \mathbf{x}^{(1)} &= \tanh \left(W^{(1)} \mathbf{x}^{(0)} + \mathbf{b}^{(1)} \right), \\ \mathbf{x}^{(2)} &= \tanh \left(W^{(2)} \mathbf{x}^{(1)} + \mathbf{b}^{(2)} \right), \\ \mathbf{x}^{(3)} &= \sigma \left(W^{(3)} \mathbf{x}^{(2)} + \mathbf{b}^{(3)} \right). \end{aligned} \quad (6)$$

For the Two-Moons classification problem, a relatively small network is sufficient:

- $W^{(1)} \in \mathbb{R}^{d_1 \times d_0}$ with $d_1 = 16$ and $d_0 = 2$.
- $W^{(2)} \in \mathbb{R}^{d_2 \times d_1}$ with $d_2 = d_1 = 16$.
- $W^{(3)} \in \mathbb{R}^{d_3 \times d_2}$ with $d_3 = 1$ and $d_2 = 16$.
- $\mathbf{b}^{(1)} \in \mathbb{R}^{d_1}$ with $d_1 = 16$.
- $\mathbf{b}^{(2)} \in \mathbb{R}^{d_2}$ with $d_2 = 16$.
- $\mathbf{b}^{(3)} \in \mathbb{R}^{d_3}$ with $d_3 = 1$.
- $\sigma^{(1)} : \mathbb{R} \rightarrow \mathbb{R}$ such that $\sigma^{(1)}(\cdot) = \tanh(\cdot)$.
- $\sigma^{(2)} : \mathbb{R} \rightarrow \mathbb{R}$ such that $\sigma^{(2)}(\cdot) = \tanh(\cdot)$.
- $\sigma^{(3)} : \mathbb{R} \rightarrow \mathbb{R}$ such that $\sigma^{(3)}(\cdot) = \sigma(\cdot)$.

Where the last activation function is the sigmoid $\sigma^{(3)} = \sigma : \mathbb{R} \rightarrow [0, 1]$:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (7)$$

Figure 2 summarizes the complete architecture together with the dimensions of every layer.

Note that a useful parameter can be presented as it will be used in the optimization section, namely the pre-activation function of a layer l $\mathbf{z}^{(l)}$:

$$\mathbf{z}^{(l)} = W^{(l)} \mathbf{x}^{(l-1)} + \mathbf{b}^{(l)} \quad (8)$$

$$\mathbf{x}^{(l)} = \sigma^{(l)}(\mathbf{z}^{(l)}) \quad (9)$$

2.3. Geometrical interpretation of the first hidden layer

Rather than immediately viewing a neural network as a black-box optimization algorithm, it is interesting to understand the geometrical role played by every

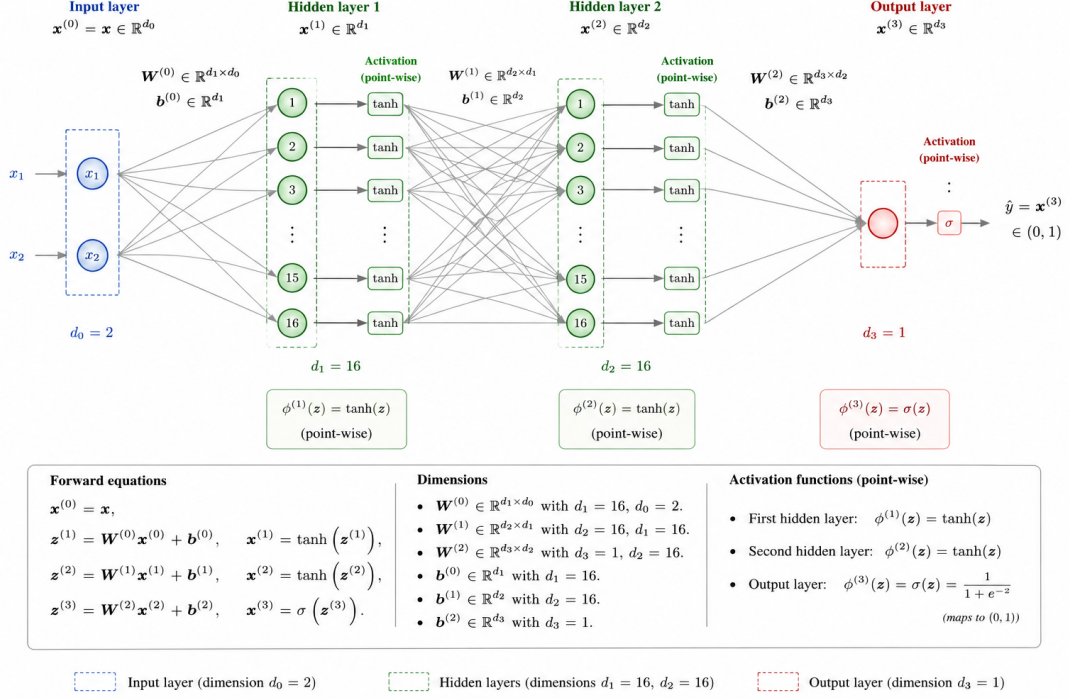


Figure 2. Architecture of the proposed Multi-Layer Perceptron.

neuron. Consider the first neuron of the first hidden layer. Before the activation function, its output is:

$$z_1^{(1)} = W_{1,1}^{(1)} \cdot x_1 + W_{1,2}^{(1)} \cdot x_2 + b_1^{(1)}$$

Equivalently:

$$z_1^{(1)} = W^{(1)}(1, :). \mathbf{x}^{(0)} + b_1^{(1)}$$

The equation:

$$W^{(1)}(1, :). \mathbf{x}^{(0)} + b_1^{(1)} = 0$$

defines an affine hyperplane. Since the input space is two-dimensional, this hyperplane is simply a line:

$$D_1 = \{(x_1, x_2) \in \mathbb{R}^2 \mid W_{1,1} \cdot x_1 + W_{1,2} \cdot x_2 + b_1 = 0\}$$

Consequently, every neuron of the first hidden layer defines its own separating line. With sixteen neurons, the first hidden layer therefore constructs sixteen different affine lines partitioning the Euclidean plane.

The quantity:

$$z_1^{(1)} = W^{(1)}(1, :). \mathbf{x}^{(0)} + b_1^{(1)}$$

helps determine on which side of the line the point lies. Indeed:

- $z_1^{(1)} > 0$ indicates that the point lies on one side of the line.

- $z_1^{(1)} < 0$ indicates the opposite side.
- $z_1^{(1)} = 0$ means that the point belongs to the line itself.

Thus, every neuron performs an elementary geometric test.

2.4. Role of the activation function

The previous affine score is then passed through the hyperbolic tangent:

$$x_1^{(1)} = \tanh(z_1^{(1)})$$

The role of the activation function is twofold. First, it introduces the nonlinearity required by the Universal Approximation Theorem. Secondly, it transforms the affine score into a bounded quantity lying inside $(-1, 1)$. Consequently:

- Values close to $+1$ indicate that the point is far from the line on the positive side.
- Values close to -1 indicate that the point is far on the opposite side.
- Values close to 0 indicate that the point is located near the separating line.

Each neuron therefore returns a normalized geometric descriptor describing the relative position of the point with respect to its associated affine line.

2.5. The feature space

After the first hidden layer, the original point $\mathbf{x} = (x_1 \ x_2)^T$ is no longer represented by its Euclidean coordinates. Instead, it is represented by the vector:

$$\mathbf{x}^{(1)} = \begin{pmatrix} x_1^{(1)} & \dots & x_{16}^{(1)} \end{pmatrix}^T \in \mathbb{R}^{16}$$

Rather than encoding Cartesian coordinates, it contains sixteen geometric tests performed by the first hidden layer.

2.6. Second hidden layer

The neurons of the second hidden layer no longer receive geometric coordinates. Instead, they receive the sixteen features computed previously. For instance:

$$z_1^{(2)} = W_{1,1}^{(1)} \cdot x_1^{(1)} + \dots + W_{1,16}^{(1)} \cdot x_{16}^{(1)} + b_1^{(1)}$$

This quantity is no longer associated with a single separating line. Instead, it combines several geometric tests simultaneously. One may interpret this neuron as learning statements such as the current point is located at the left of D_2 , but above D_3 et cetera. The second hidden layer therefore combines elementary geometric information into increasingly complex spatial patterns. Unlike the first hidden layer, which partitions the plane into half-planes, the second hidden layer constructs nonlinear decision regions through combinations of these elementary tests.

2.7. Output layer

Finally, the last layer combines all previously extracted features. Its pre-activation output is:

$$\mathbf{z}^{(3)} = W^{(2)} \cdot \mathbf{x}^{(2)} + \mathbf{b}^{(2)} \quad (10)$$

Applying the sigmoid function yields:

$$\mathbf{x}^{(3)} = \sigma(\mathbf{z}^{(3)}) \quad (11)$$

Since:

$$\sigma : \mathbb{R} \rightarrow [0, 1]$$

the output naturally admits a probabilistic interpretation. For example, $\mathbf{f}_\theta(\mathbf{x}) = 0.98$ indicates that the network estimates a probability of 98% that the point belongs to the red class.

At this stage, however, these probabilities remain meaningless because the parameters in θ have only been randomly initialized. The purpose of the training procedure is precisely to adjust these parameters

through optimization so that the geometric structures learned by the successive layers gradually evolve into an accurate nonlinear decision boundary separating the two moons.

2.8. Final classification step

In fact, the output of the last layer is a probability value comprised between 0 and 1. However, as specified in the beginning, our goal is to have a boolean value at the output of our NN, allowing us to decide whether the point belongs to a part of the moon or not. For that, we employ a prediction threshold $p_t \in [0, 1]$ such that if $x^{(3)} > p_t$, then: $\mathbf{f}_\theta(\mathbf{x}) = 1$ and the point belongs to a certain part of the moons. And if $x^{(3)} \leq p_t$, then: $\mathbf{f}_\theta(\mathbf{x}) = 0$ and the point belongs to the other quadrant of the moon. In our study $p_t = 0.5$.

3. TRAINING OF THE NN

The goal now is to train the NN to find the best parameters composing the vector θ . As **supervised learning** is used to train our model, a set $D = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$ is given as input with known data pairs.

Therefore, provided a known input $\mathbf{x}^{(i)}$ vector in $\mathbb{R}^2$, the output of a forward pass of the NN will allow to compute $\mathbf{f}_\theta(\mathbf{x}^{(i)}) = \hat{y}$. Training the NN in the best way we can means finding the optimal parameters θ , that is, the weights W and biases $\mathbf{b}$ for each layer.

3.1. Optimality of the NN and cost function

Optimization involves the minimization or maximization of a certain function under specific constraints using a dedicated algorithm. In our case, the objective is to start from $D = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$ to build a function $\mathbf{f}_\theta(\mathbf{x})$ that **works well on unseen data**, allowing to predict if a new point belongs to the red or blue part of the moon. Such a function is not continuous, and the optimization algorithm might struggle to optimize the chosen performance index. To avoid this, we replaced it using a probability in our NN. For example, if the success criteria is to have a red point and if $\mathbf{f}_\theta(\mathbf{x}) = 0.93 = 93\%$, it means that the model estimates that the given input point is red at 93%. On that case, we note that:

$$\mathbf{f}_\theta(\mathbf{x}) = \mathbb{P}(Y = 1 | X = \mathbf{x}) \quad (12)$$

Which represents the conditional probability of obtaining the successful classification in output of our model given a new input value. For several independent observations i , the likelihood of

obtaining a successful label-coherent result from our NN is modeled through:

$$\mathcal{L}(\theta) = \prod_{i=1}^N \mathbb{P}_{\theta}(Y = y^{(i)} | X = \mathbf{x}^{(i)}) \quad (13)$$

The purpose of the optimization is to maximize this quantity. Indeed, if we consider that for an observation i , the following labeled pairs is given: $(\mathbf{x}, 1)$. Suppose that for the currently trained NN with parameters θ_1 computes $f_{\theta_1}(\mathbf{x}) = 0.2 = 2\%$. Then, the model tells that there are 2% chances for the point to be red, which is a bad estimation since the labeled input is 1; meaning that the point is red. Suppose that for another trained model $f_{\theta_2}(\mathbf{x}) = 0.96 = 96\%$, this time, this NN explains the dataset in a better way.

In our case, since $Y = 1$ or $Y = 0$, it is straightforward to note that the random variable $Y|X$ follows a Bernoulli law of parameter $f_{\theta}(\mathbf{x}^{(i)})$:

$$Y | X = \mathbf{x}^{(i)} \sim \mathcal{B}(f_{\theta}(\mathbf{x}^{(i)})) \quad (14)$$

According to this assumption, equation 13 can be further developed based on the fact that:

$$\mathbb{P}_{\theta}(Y = y^{(i)} | X = \mathbf{x}^{(i)}) = (\hat{y}^{(i)})^{y^{(i)}} \cdot (1 - \hat{y}^{(i)})^{1-y^{(i)}} \quad (15)$$

Leading to:

$$\mathcal{L}(\theta) = \prod_{i=1}^N (\hat{y}^{(i)})^{y^{(i)}} \cdot (1 - \hat{y}^{(i)})^{1-y^{(i)}} \quad (16)$$

It is known that the optimization solver handles summation better than products. Therefore, one can define the log-likelihood as follows:

$$\mathcal{L}_L(\theta) = \log\left(\prod_{i=1}^N (\hat{y}^{(i)})^{y^{(i)}} \cdot (1 - \hat{y}^{(i)})^{1-y^{(i)}}\right) \quad (17)$$

Which leads after simplification, to the following classical Binary Cross-Entropy (BCE) cost function to be maximized:

$$\mathcal{L}_L(\theta) = \sum_{i=1}^N y^{(i)} \cdot \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \cdot \log(1 - \hat{y}^{(i)}) \quad (18)$$

3.2. Optimization and Backpropagation

As a reminder, the optimal tuning of the weights and bias, components of the optimization vector in equation 5, will be used to try and search for the maximal possible value of the log-likelihood using

an optimization solver.

For that, an iteration on the weights and biases can be realized using the concept of Stochastic Gradient Descent (SGD). The algorithm is based on the notion that the direction of the steepest slope is given by the negative of the gradient [4]. In a more rigorous note, if the multi-variable function $\mathcal{L}_L(\theta)$ is defined and differentiable in a neighborhood of a point θ_p , then $\mathcal{L}_L(\theta)$ decreases fastest if one goes from θ_p to a direction of negative gradient $-\nabla \mathcal{L}_L(\theta_p)$. This results in the following update rule and discretized algorithm:

$$\theta_{p_{k+1}} = \theta_{p_k} - \eta \cdot \nabla_{\theta} \mathcal{L}_L(\theta_{p_k}) \quad (19)$$

In other words, the weights and biases of the optimization vector shall be updated according to $\forall l = \{1, \dots, L\}$:

$$W_{k+1}^{(l)} = W_k^{(l)} - \eta \cdot \nabla_{W_k} \mathcal{L}_L(W_k) = W_k^{(l)} - \eta \cdot \frac{\partial \mathcal{L}_L(W_k)}{\partial W_k^{(l)}} \quad (20)$$

$$b_{k+1}^{(l)} = b_k^{(l)} - \eta \cdot \nabla_{b_k} \mathcal{L}_L(b_k) = b_k^{(l)} - \eta \cdot \frac{\partial \mathcal{L}_L(b_k)}{\partial b_k^{(l)}} \quad (21)$$

The challenge is therefore to compute the partial derivatives with respect to the weight matrices and bias vectors. For that, we made use of the Chain's rule. Suppose that g is differentiable on an open U of $\mathbb{R}^n$ and that it exists:

$$\forall i = [1, n], u_i : \mathbb{R} \rightarrow I \subset \mathbb{R}$$

Such that:

$$\forall t \in I, (u_1(t) \quad \dots \quad u_n(t)) \in U$$

Then, the **Chaine's rule** states that:

$$(g(u_1(t), \dots, u_n(t)))' = \sum_{i=1}^n u_i(t)' \cdot \frac{\partial g}{\partial x_i}(u_1(t), \dots, u_n(t)) \quad (22)$$

Leading to the classical result with two composed functions of a single variable, if $n = 1$:

$$(g(u_1(t)))' = u_1(t)' \cdot \frac{\partial g}{\partial x_1}(u_1(t)) = u_1(t)' \cdot \frac{\partial g}{\partial t}(u_1(t)) \quad (23)$$

Or, if $u_1 = f$, to:

$$(g \circ f)'(t) = f'(t) \cdot g'(f(t)) \quad (24)$$

The following general algorithm is the one implemented from scratch for the training of the NN [4]:

Algorithm 1 Stochastic Gradient Descent and Backpropagation for the Two-Moons MLP

Require: Dataset $\mathcal{D}_N = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$, learning rate η , number of epochs T , initial parameters $\theta = \{W^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^3$

Ensure: Trained parameters θ^*

```

1: for  $t = 1, \dots, T$  do
2:   Randomly select  $i \in \{1, \dots, N\}$ 
3:    $\mathbf{x}^{(0)} \leftarrow \mathbf{x}^{(i)}$ 
                                     ▷ Forward pass
4:   for  $l = 1, \dots, 3$  do
5:      $\mathbf{z}^{(l)} \leftarrow W^{(l)}\mathbf{x}^{(l-1)} + \mathbf{b}^{(l)}$ 
6:     if  $l = 1$  or  $l = 2$  then
7:        $\mathbf{x}^{(l)} \leftarrow \tanh(\mathbf{z}^{(l)})$ 
8:     else
9:        $\mathbf{x}^{(3)} \leftarrow \sigma(\mathbf{z}^{(3)})$ 
10:    end if
11:  end for
12:   $\hat{y}^{(i)} \leftarrow \mathbf{x}^{(3)}$ 
13:   $\mathcal{L}^{(i)}(\theta) \leftarrow -[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$ 
                                     ▷ Backward pass
14:   $\delta^{(3)} \leftarrow \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{x}^{(3)}} \odot \sigma'(\mathbf{z}^{(3)})$ 
15:  for  $l = 2, \dots, 1$  do
16:     $\delta^{(l)} \leftarrow (W^{(l+1)})^T \delta^{(l+1)} \odot (1 - \tanh^2(\mathbf{z}^{(l)}))$ 
17:  end for
                                     ▷ Gradient computation
18:  for  $l = 1, \dots, 3$  do
19:     $\frac{\partial \mathcal{L}^{(i)}}{\partial W^{(l)}} \leftarrow \delta^{(l)} (\mathbf{x}^{(l-1)})^T$ 
20:     $\frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{b}^{(l)}} \leftarrow \delta^{(l)}$ 
21:  end for
                                     ▷ SGD update
22:  for  $l = 1, \dots, 3$  do
23:     $W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial \mathcal{L}^{(i)}}{\partial W^{(l)}}$ 
24:     $\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{b}^{(l)}}$ 
25:  end for
26: end for
27: return  $\theta^* = \{W^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^3$ 

```

The training phase consists in a backpropagation loop, where the generated data contained in D as ground truth is used to optimize the weights and biases of the NN to minimize the selected cost function at the end. Since the NN is built from scratch, even the SGD solver has been coded. For that, the following matrix calculus has been developed and implemented. The goal is to retrieve the following partial derivatives, where indexes (i) and k have been dropped for clarity:

$$\frac{\partial \mathcal{L}_L(W_k)}{\partial W_k^{(l)}}, \frac{\partial \mathcal{L}_L(b_k)}{\partial b_k^{(l)}}$$

We will focus our attention to the first term since the second one is obtained using the same methodology. Note that the partial derivative have a special expression in function of the considered layer. We will divide our derivation of this expression for the

last layer L , then present the general case of layers $l < L$. The Chaine rule allows us to notice that for the last layer L :

$$\frac{\partial \mathcal{L}_L(W)}{\partial W^{(L)}} = \frac{\partial \mathcal{L}_L(W)}{\partial \mathbf{z}^{(L)}} \cdot \frac{\partial \mathbf{z}^{(L)}}{\partial W^{(L)}} \quad (25)$$

Based on equations 8 and 9, we can note that:

$$\frac{\partial \mathbf{z}^{(L)}}{\partial W^{(L)}} = \mathbf{x}^{(L-1)} \quad (26)$$

Concerning first term of the product in the right-hand side of equation 25, we can settle the general case for the layer l to be:

$$\frac{\partial \mathcal{L}_L}{\partial \mathbf{z}^{(l)}} \triangleq \delta^{(l)} \quad (27)$$

Which will be very useful in our calculations. A clear dependence of the layer l to the layer $l + 1$ can be exhibited through this parameter, in terms of weights and bias optimization backpropagation. Indeed, based on equation 8:

$$\mathbf{z}^{(l+1)} = W^{(l+1)} \cdot \mathbf{x}^{(l)} + \mathbf{b}^{(l+1)}$$

On the other hand, using Chaine's rule and the definition established through equation 27:

$$\frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{z}^{(l)}} = \frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathcal{L}_L} \cdot \frac{\partial \mathcal{L}_L}{\partial \mathbf{z}^{(l)}} = \frac{\delta^{(l)}}{\delta^{(l+1)}} = \frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{x}^{(l)}} \cdot \frac{\partial \mathbf{x}^{(l)}}{\partial \mathbf{z}^{(l)}}$$

Leading to, $\forall l < L$:

$$\delta^{(l)} = \delta^{(l+1)} \cdot W^{(l+1)} \cdot \sigma'(\mathbf{z}^{(l)}) \quad (28)$$

This expression is valid for any layer placed ahead of the last one, since if $l = L$, then $L + 1$ is not defined and $W^{(L+1)}$ does not exist. Therefore, the special case of the last layer is treated as follows. Since:

$$\frac{\partial \mathcal{L}_L}{\partial \mathbf{z}^{(L)}} = \frac{\partial \mathcal{L}_L}{\partial \mathbf{x}^{(L)}} \cdot \frac{\partial \mathbf{x}^{(L)}}{\partial \mathbf{z}^{(L)}}$$

One can easily see that:

$$\frac{\partial \mathbf{x}^{(L)}}{\partial \mathbf{z}^{(L)}} = \sigma'(\mathbf{z}^{(L)})$$

And that:

$$\frac{\partial \mathcal{L}_L}{\partial \mathbf{x}^{(L)}} = \hat{y} - y \quad (29)$$

Again, this is obtained using the Chaine's rule since $\mathbf{x}^{(L)} = \mathbf{x}^{(3)} = \mathbf{z}^{(L=3)}$, hence:

$$\frac{\partial \mathcal{L}_L}{\partial \mathbf{x}^{(L)}} = \frac{\partial \mathcal{L}_L}{\partial \mathbf{z}^{(L)}} = \frac{\partial \mathcal{L}_L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial \mathbf{z}^{(L)}}$$

The first term of the product in the right-hand side of the previous equation is easily retrievable from the BCE's equation 18:

$$\frac{\partial \mathcal{L}_L}{\partial \hat{y}} = \frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}}$$

Whereas the second term is:

$$\frac{\partial \hat{y}}{\partial z^{(L)}} = \frac{\partial \mathbf{x}^{(L)}}{\partial z^{(L)}} = \sigma'^{(l)}(\mathbf{z}^{(L)})$$

That can be further developed into:

$$\frac{\partial \sigma(\mathbf{z}^{(L)})}{\partial z^{(L)}} = \frac{\partial}{\partial z^{(L)}} \left(\frac{1}{1 + e^{-z^{(L)}}} \right) = \frac{e^{-z^{(L)}}}{(1 + e^{-z^{(L)}})^2}$$

Resulting in:

$$\frac{\partial \sigma(\mathbf{z}^{(L)})}{\partial z^{(L)}} = \sigma(\mathbf{z}^{(L)}) \cdot (1 - \sigma(\mathbf{z}^{(L)})) \quad (30)$$

Eventually, this leads to the following partial derivative parameters for layer L :

$$\frac{\partial \mathcal{L}_L(W)}{\partial W^{(L)}} = \mathbf{x}^{(L-1)} \cdot (\delta^{(L)})^T \quad (31)$$

$$\delta^{(L)} = (\hat{y} - y) \cdot \sigma'^{(l)}(\mathbf{z}^{(L)}) \quad (32)$$

The conclusion for layers $l < L$ are quite similar:

$$\frac{\partial \mathcal{L}_L(W)}{\partial W^{(l)}} = \mathbf{x}^{(l-1)} \cdot (\delta^{(l)})^T \quad (33)$$

$$\delta^{(l)} = \delta^{(l+1)} \cdot W^{(l+1)} \cdot \sigma'^{(l)}(\mathbf{z}^{(l)}) \quad (34)$$

This closes the mathematical justification behind our implemented self-made solver.

4. IMPLEMENTED ALGORITHMS

Two scripts have been developed for implement the NN for the Two-Moons classification problem. The first one, designed from scratch without any Pytorch, TensorFlow, or Keras libraries is intended to deeply understand the functioning and underlying mathematics of an MLP. It can be accessed through the following link [Two-Moons from scratch](#).

On the other hand, the equivalent script have also been built based on usual object-oriented definition of classes in Python and using classical ML libraries. The project can be accessed through [Two-Moons classical ML libraries](#).

The obtained results are quite similar for the two scripts. For the first one, the generated Two-Moons data is as follows:

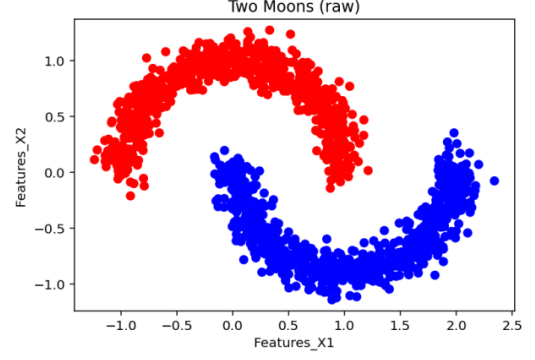


Figure 3. Two-Moons ground-truth data generation NN from scratch.

The algorithm returns a test accuracy of around 95%, exhibiting a good generalization to unseen data:

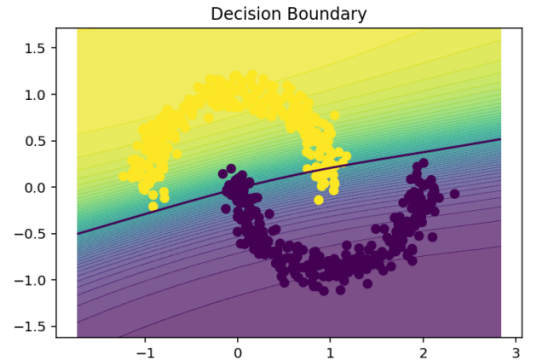


Figure 4. Two-Moons test results.

Note that the data generation has been done using the Python script `Make_Two_Moons_Dataset.py`. The test data comes from the general generated data sample pool, where around 60% of the total of 1500 pairs is kept for training and the rest for testing. 600 Epochs were considered for the training.

Keeping the same numerical values and using the classical ML architectures and libraries leads to the following tests results:

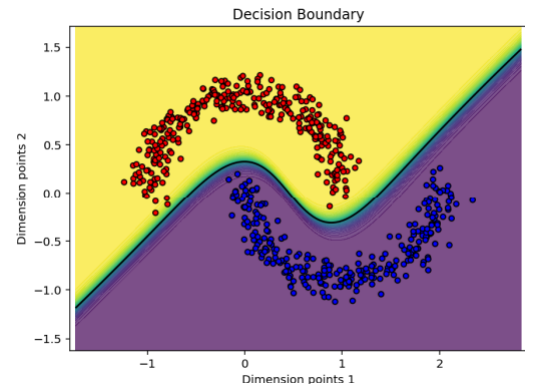


Figure 5. Two-Moons test results classical ML libraries.

Again, exhibiting a very good accuracy of around

100%.

5. CONCLUSION

This first article introduced the mathematical foundations of a classical MLP through the well-known Two-Moons classification problem. Rather than relying on existing Machine Learning libraries, every component of the neural network was implemented from scratch, including the forward propagation, the BCE loss, the backpropagation algorithm and the SGD optimization procedure. This approach provided a complete understanding of the mathematical mechanisms governing the training of a neural network, from the geometrical interpretation of individual neurons to the optimization of the trainable parameters.

The obtained results demonstrate that even a relatively small fully-connected neural network is capable of learning a highly nonlinear decision boundary, leading to an very good classification accuracy on unseen data. This confirms, on a simple benchmark, the approximation capabilities predicted by the Universal Approximation Theorem.

However, although such architectures are remarkably efficient for approximating finite-dimensional mappings

$$f_{\theta} : \mathbb{R}^d \rightarrow \mathbb{R}^m,$$

their direct application to scientific computing rapidly reaches important limitations. In particular, when solving parameterized Partial Differential Equations, a classical neural network generally learns a single solution associated with a specific parameter configuration. Consequently, modifying the coefficients or the initial or boundary conditions typically requires retraining the entire network, making this approach computationally expensive and poorly suited for many-query scientific problems.

These limitations naturally motivate the introduction of Neural Operators. Rather than approximating a single function, Neural Operators learn mappings between infinite-dimensional function spaces, allowing an entire family of PDE solutions to be approximated within a single trained model. The next article of this series will therefore introduce the theoretical foundations of Operator Learning before presenting the Fourier Neural Operator (FNO), which constitutes one of the first practical architectures capable of efficiently learning solution operators for parameterized PDEs.

Conflict of Interest: The author declares no conflict of interest.

Author Contributions: Author 1 contributed to conceptualization, methodology, and writing.

Funding: This research received no external funding.

Ethical Statement: This study follows ethical and academic integrity guidelines.

6. REFERENCES

- [1] N. Kovachki and L. Zongyi, “Neural operator: Learning maps between function spaces with applications to pdes,” *Journal of Machine Learning Research*, 2022.
- [2] L. Zongyi, “Physics-informed neural operator for learning partial differential equations,” *ACM / IMS Journal of Data Science*, 2023.
- [3] P. Petersen and J. Zech, *Mathematical theory of deep learning*, Available: <https://arxiv.org/abs/2407.18384>, 2023.
- [4] L. e. a. Kaelbling, *Mathematical theory of deep learning*, Available: <https://ocw.mit.edu/>, 2019.